CHAPTER 02: TESTING THROUGHOUT THE SDLC

1. Core Syllabus Concepts & In-Depth Notes

2.1 Testing in the Context of a Software Development Lifecycle (SDLC)

> **Formal Definition:** An **SDLC model** is an abstract, high-level representation of the software development process. It defines how different development phases and activities relate to each other, both *logically* and *chronologically*.

2.1.1 Impact of the SDLC on Testing (FL-2.1.1 — K2)

Testing must be adapted to the chosen SDLC to succeed. The choice of SDLC directly dictates: (1) Scope and timing of test activities, (2) Level of detail of test documentation, (3) Choice of test techniques and approaches, (4) Extent of test automation, and (5) Roles and responsibilities of testers.

SDLC Category	Model Examples	Testing Characteristics & Dynamics
Sequential Models	Waterfall Model, V-Model	• Testers participate early in requirements reviews, test analysis, and test design. • Executable code is created in later phases; dynamic testing cannot start early. • Strict phase gates: exit criteria of one phase often form entry criteria for the next.
Iterative & Incremental Models	Spiral Model, Prototyping, Unified Process (UP)	• Each iteration delivers a working prototype or product increment. • Static and dynamic testing are performed across all test levels in every iteration. • Requires rapid feedback and extensive regression testing as features accumulate.
Agile Software Development	Scrum, Kanban, Extreme Programming (XP), Lean IT, FDD	• Assumes continuous change throughout the project lifecycle. • Lightweight work product documentation favored over heavy formal test plans. • Heavy reliance on automated regression testing and CI/CD pipelines. • Manual testing relies heavily on experience-based techniques (exploratory testing).

2.1.2 Software Development Lifecycle and Good Testing Practices `FL-2.1.2 (K1)`

Regardless of which SDLC model an organization adopts, four universal testing practices must always be implemented:

1. Corresponding Test Activities
For **every** software development activity, there is a corresponding test activity, ensuring that all development work products are subject to quality control.

2. Distinct Test Level Objectives
Different test levels have specific and different test objectives, enabling comprehensive testing while avoiding wasteful duplication and redundancy.

3. Early Test Analysis & Design
Test analysis and design for a given test level begin during the corresponding development phase of the SDLC (adhering to Principle 3: Early Testing).

4. Early Review Involvement (Shift Left)
Testers are involved in reviewing work products as soon as early drafts are available, detecting defects before implementation begins.

2.1.3 Testing as a Driver for Software Development FL-2.1.3 (K1)

In test-first approaches, tests are defined **before** the code is written. This embodies the principle of early testing and shift-left, directing development and supporting iterative lifecycles.

Approach	Core Mechanism & Definition	Academic Citation & Format
Test-Driven Development (TDD)	Directs coding through test cases rather than extensive upfront software design. Developer writes an automated unit test → writes minimal code to pass → refactors code and tests.	**Beck (2003)** *Cycle: Red → Green → Refactor*
Acceptance Test-Driven Development (ATDD)	Derives tests from customer acceptance criteria during system design. Tests are created collaboratively by business, developers, and testers before features are coded.	**Gärtner (2011)** *Collaborative Specification Workshops*
Behavior-Driven Development (BDD)	Expresses application behavior using simple natural language scenarios that all stakeholders understand. Scenarios are automatically translated into executable tests.	**Chelimsky (2010)** *Given / When / Then format*

Persistence as Automated Regression Tests: In all three approaches (TDD, ATDD, BDD), the authored tests persist as an automated test suite that guards against regressions during future code modifications and refactoring.

2.1.4 DevOps and Testing FL-2.1.4 (K2)

DevOps is an organizational approach creating synergy by getting development (including testing) and operations to collaborate toward common goals (Kim 2016). It represents a cultural shift bridging silos with shared accountability, continuous integration (CI), continuous delivery (CD), and automated deployment pipelines.

Testing Benefits in DevOps:

- **Fast Quality Feedback:** Immediate visibility into code health and whether changes break existing functionality.
- **CI Promotes Shift Left:** Encourages developers to commit high-quality code accompanied by component tests and static analysis.
- **Stable Test Environments:** Automated CI/CD provisioning creates reliable, reproducible test environments on demand.
- **Non-Functional Visibility:** Early automated checks for performance efficiency, reliability, and security in the pipeline.
- **Reduced Manual Burden:** Automating repetitive regression execution frees testers for high-value exploratory testing.
- **Minimized Regression Risk:** Comprehensive test suites run continuously on every commit.

DevOps Risks & Challenges:

- **Pipeline Complexity:** Establishing and maintaining delivery pipelines requires substantial technical infrastructure.
- **Toolchain Overhead:** Integrating and managing diverse CI/CD tools demands specialized expertise.
- **Automation Maintenance:** Creating and maintaining automated test suites is resource-intensive and expensive.
- **Manual Testing Still Vital:** Manual testing—especially from the end-user's perspective (usability, exploratory)—remains essential.

2.1.5 Shift Left FL-2.1.5 (K2)

Concept: "Shift Left" is the practical implementation of **Principle 3 (Early Testing saves time and money)**. It means performing testing earlier in the SDLC timeline, rather than waiting for code implementation or component integration. *It does NOT mean neglecting testing later in the SDLC!*

Five Good Practices to Achieve Shift Left:

1. Reviewing Specifications Early
Testers review requirements, user stories, and specifications to identify ambiguities, omissions, and contradictions before development begins.

2. Authoring Tests Before Code (TDD/ATDD)
Writing test cases prior to implementation so code can be executed against test harnesses during development.

3. Automated CI/CD Testing
Using Continuous Integration to provide instant feedback with automated component tests whenever code is committed.

4. Early Static Code Analysis
Running automated static analysis on source code prior to dynamic testing to detect code complexity and vulnerabilities.

5. Early Non-Functional Testing at Component Level:
Conducting performance efficiency and security tests early at the component level rather than waiting for a fully assembled system in late staging environments.

Shift-Left Trade-Off & Stakeholder Buy-In: Shift left requires upfront investment in training, tooling, and effort earlier in the lifecycle, but generates massive downstream cost savings. Stakeholders must be convinced and bought into this concept.

2.1.6 Retrospectives and Process Improvement FL-2.1.6 (K2)

Retrospectives (often held at sprint ends, release milestones, or post-project) bring together cross-functional participants (testers, developers, architects, product owner, business analysts) to drive **continuous improvement**.

1. What was successful, and should be retained?	2. What was not successful, and could be improved?	3. How to incorporate improvements in the future?

Five Key Benefits of Retrospectives for Testing:

- **Increased Test Effectiveness & Efficiency:** Implementing agreed process improvements streamlines testing workflows.
- **Higher Quality Testware:** Jointly reviewing testing practices leads to better test cases, test data, and scripts.
- **Team Bonding & Psychological Safety:** Provides an open forum to discuss challenges constructively without blame.

- **Improved Quality of the Test Basis:** Deficiencies in user stories or requirements are identified and resolved upstream.
- **Enhanced Collaboration:** Strengthens mutual trust and empathy between developers and testers.

*Note: Retrospective findings and action items are officially recorded in the **Test Completion Report**.*

1.2 Test Levels and Test Types

2.2.1 The Five Test Levels FL-2.2.1 (K2)

> **Formal Definition: Test levels** are groups of test activities that are organized and managed together. Each level is an instance of the test process performed in relation to software at a given development phase (from unit to systems of systems).

1. Component Testing
2. Component Integration
3. System Testing
4. System Integration
5. Acceptance Testing

Test Level	Test Object & Basis	Focus & Objectives	Typical Responsibilities & Tools
Component Testing (Unit Testing)	**Object:** Components, units, modules, classes, code. **Basis:** Detailed design, component specs, code.	Focuses on testing components in isolation. Verifies functional logic, data flows, and code structure. Finds code defects, calculation errors.	**Who:** Developers in dev environment. **Tools:** Unit test frameworks (JUnit, NUnit), mock frameworks, test harnesses.
Component Integration Testing	**Object:** Interactions between components, APIs, database interfaces. **Basis:** Software architecture, interface design.	Focuses on interfaces and communications between components. Detects interface mismatches, incorrect data encoding, communication protocol flaws.	**Who:** Developers / Technical testers. **Approaches:** Top-down, bottom-up, big-bang (risky).
System Testing	**Object:** Entire integrated product/system. **Basis:** System/software requirements, use cases, functional specs.	Evaluates end-to-end tasks, overall capabilities, business flows, and non-functional quality characteristics against system specifications.	**Who:** Independent test team. **Environment:** Dedicated, representative test environment.
System Integration Testing	**Object:** Interactions between system under test and other external systems/services. **Basis:** System architecture, external interface specs.	Focuses on interfaces between systems, microservices, cloud APIs, external third-party services, hardware interfaces.	**Who:** Dedicated test team / integration engineers. **Environment:** Close to operational environment.
Acceptance Testing	**Object:** Complete system, business processes, user workflows. **Basis:** Business requirements, user stories, contracts, regulations.	Focuses on **validation** and demonstrating readiness for deployment. Checks whether the system satisfies business goals and user needs.	**Who:** Intended end users, customers, system operators, business representatives.

Five Distinguishing Attributes of Test Levels (Avoids Overlap):

1. **Test Object** (what is tested) • 2. **Test Objectives** (goals) • 3. **Test Basis** (reference specs) • 4. **Defects and Failures** (types of flaws expected) • 5. **Approach and Responsibilities** (who tests and how).

Forms of Acceptance Testing (FL-2.2.1)

1. User Acceptance (UAT)

Intended business users evaluate the system to confirm it supports everyday business processes.

2. Operational Acceptance (OAT)

Sysadmins test backup/restore, disaster recovery, failover, maintenance tasks, and security.

3. Contractual & Regulatory

Verifies custom software meets contract terms or complies with statutory laws and safety standards.

4. Alpha Testing:

Performed at the **developer's site** by potential users or independent test teams not involved in development.

5. Beta Testing:

Performed by potential or existing customers at their **own operational locations** in real-world conditions.

2.2.2 The Four Test Types FL-2.2.2 (K2)

Formal Definition: A **test type** is a group of test activities related to specific quality characteristics. *Crucial rule: All four test types can be performed at EVERY test level!*

Test Type	Core Focus & Main Objectives	Key Details & Standards
Functional Testing	Evaluates functions the system should perform (**"WHAT"** the test object does). Checks completeness, correctness, and appropriateness.	Specification-based. Can be executed at all test levels (e.g. unit calculation, system checkout flow).
Non-Functional Testing	Evaluates **"HOW WELL"** the system behaves. Measures quality attributes against defined constraints.	Based on **ISO/IEC 25010**. Often derived from functional tests adding constraints (e.g. execution speed, load).
Black-Box Testing	Specification-based. Tests derived without reference to internal code structure. Checks behavior against specs.	Test cases remain valid even if internal code implementation is refactored (as long as specs don't change).
White-Box Testing	Structure-based. Tests derived from internal implementation (code, architecture, data/control flows).	Measures objective structural code coverage (statements, branches). Created after design or implementation.

ISO/IEC 25010 (2023) Quality Characteristics for Non-Functional Testing:

1. Performance Speed, response time, resource utilization, throughput.	**2. Compatibility** Co-existence and interoperability with other systems.	**3. Usability** Interaction capability, learnability, error protection.	**4. Reliability** Maturity, availability, fault tolerance, recoverability.
5. Security Confidentiality, integrity, non-repudiation, authentication.	**6. Maintainability** Modularity, reusability, analyzability, modifiability, testability.	**7. Portability** Flexibility, adaptability, installability, replaceability.	**8. Safety** Absence of unacceptable risk of harm to people or environment.

2.2.3 Confirmation Testing and Regression Testing FL-2.2.3 (K2)

When software changes to fix defects or add features, both confirmation and regression testing must be performed:

Dimension	Confirmation Testing (Re-testing)	Regression Testing
Objective	Confirms that an original defect has been successfully fixed and the failure no longer occurs.	Confirms that no adverse side effects have been caused by a change in unchanged parts of the software.
Scope	Targeted: re-executing failed tests and tests covering bug-fix modifications.	Broad: unchanged features in the same component, other subsystems, or connected environments.
Tester Assignment	**Best practice:** Done by the *same person* who performed the initial test and logged the defect.	Can be executed by any tester or run automatically via CI/CD test automation pipelines.
Automation Role	Often executed manually or via targeted unit tests.	**Strongest candidate for automation** because regression suites grow and run repeatedly.

> **Impact Analysis in Regression Testing:** Prior to regression testing, testers should perform **impact analysis** to identify all areas of the software and environment potentially affected by the modification, scoping tests efficiently.

2.3 Maintenance Testing `FL-2.3.1 (K2)`

Maintenance testing evaluates changes to an operational system or checks for regressions in unchanged parts (which usually constitute most of the system). Governed by **ISO/IEC/IEEE 14764** (corrective, adaptive, perfective maintenance).

What Determines the Scope of Maintenance Testing? (3 Factors):

1. Risk of the Change
Criticality of modified component and potential financial/safety impact of failure.

2. Size of Existing System
Larger systems possess complex interdependencies requiring wider regression checks.

3. Size of the Change
Minor emergency bug fix vs complete architectural microservice overhaul.

The Three Triggers for Maintenance Testing:

Maintenance Trigger	Real-World Practical Scenarios	Required Testing Focus
1. Modifications	Planned enhancements, new user stories, corrective bug fixes, emergency production hot fixes.	Confirmation testing of fixes + regression testing of affected and related components.
2. Upgrades or Migrations	Migrating software to a new OS, cloud host, database engine, or migrating legacy database records into a new schema.	Environment compatibility tests, data conversion verification, and operational checks.
3. Retirement	Decommissioning an application at the end of its operational lifecycle.	Testing data archiving procedures, retention compliance, and data restore/retrieval capabilities.

2. High-Yield Exam Flashpoints & Key Distinctions

TOP 10 ISTQB CTFL v4.0.1 CHAPTER 2 EXAM TRAPS & DISTRACTORS:

1. **"Component Integration and System Integration are the same test level"** → **FALSE!** v4.0.1 explicitly splits them. Component Integration tests interfaces between code units; System Integration tests interfaces between independent systems and external services.
2. **"Non-functional testing can only be performed at System Testing"** → **FALSE!** Non-functional testing (e.g. performance efficiency, security) can and should be conducted at *every* test level, including component testing (Shift Left).
3. **"White-box testing is only used at the component test level"** → **FALSE!** White-box techniques can be applied at integration and system levels (e.g. business process workflows, API call flows).
4. **"Confirmation testing and regression testing are synonyms"** → **FALSE!** Confirmation testing verifies that a specific defect is fixed; regression testing checks that unchanged code didn't break.
5. **"Regression testing should only be done manually"** → **FALSE!** Regression test suites grow over time and run repeatedly, making them the strongest candidate for automation in CI/CD.
6. **"DevOps eliminates the need for manual testing"** → **FALSE!** Although DevOps relies heavily on automation, manual testing (especially user-oriented usability and exploratory) is still essential.
7. **"Shift-left means eliminating testing late in the SDLC"** → **FALSE!** Shift-left introduces testing earlier (e.g., reviews, pre-code tests), but does not replace or neglect system/acceptance testing.
8. **"Acceptance testing must always be performed by independent testers"** → **FALSE!** Acceptance testing should ideally be performed by intended business users, customers, or system operators.
9. **"Retrospectives are only attended by testers"** → **FALSE!** Retrospectives require cross-functional attendance: developers, testers, architects, product owners, and business analysts.
10. **"Maintenance testing only occurs after emergency bugs"** → **FALSE!** Maintenance testing is triggered by planned modifications, environment migrations/upgrades, and system retirement.

Rapid Comparison Matrix (Core Syllabic Contrasts)

Concept A	Concept B	Crucial Distinguishing Test
Component Integration	**System Integration**	Component Integration tests interactions between *internal modules*; System Integration tests interactions with *external systems/APIs*.
Alpha Testing	**Beta Testing**	Alpha testing occurs at the *developer's site*; Beta testing occurs at the *customer's own location*.
User Acceptance (UAT)	**Operational Acceptance (OAT)**	UAT validates *business workflows*; OAT validates *operational readiness (backup, restore, disaster recovery)*.
TDD	**ATDD**	TDD directs *unit code implementation*; ATDD directs *feature acceptance based on user acceptance criteria*.
Functional Testing	**Non-Functional Testing**	Functional tests *"WHAT the system does"*; Non-Functional tests *"HOW WELL the system behaves"*.

High-Yield Memory Mnemonics for Revision:

- **5 Test Levels:** *Component → Component Integration → System → System Integration → Acceptance*.
- **4 Test Types:** *Functional ("What") | Non-Functional ("How Well") | Black-Box (Spec) | White-Box (Structure)*.
- **3 Maintenance Triggers:** *Modifications • Migrations • Retirement*.
- **3 Test-First Approaches:** *TDD (Code-first) | ATDD (Criteria-first) | BDD (Behavior Given/When/Then)*.

3. Complete Chapter 2 Keyword Lexicon (All 17 Official Terms)

Acceptance Testing A test level that focuses on determining whether to accept the system, validating readiness for deployment and meeting business needs.

Black-Box Testing Testing, either functional or non-functional, without reference to the internal structure of the component or system.

Component Integration Testing Testing the interfaces and interactions between integrated components in isolation from the rest of the system.

Component Testing Testing individual hardware or software components in isolation (also known as unit testing).

Confirmation Testing (Re-testing) Dynamic testing conducted after a defect fix to confirm that the failure caused by the original defect does not recur.

Functional Testing Testing conducted to evaluate the compliance of a component or system with functional requirements ("what it does").

Integration Testing Testing designed to discover defects in the interfaces and interactions between components or systems.

Maintenance Testing Testing the changes to an operational system or the impact of changes to the operational environment.

Non-Functional Testing Testing conducted to evaluate quality characteristics other than functionality ("how well the system behaves").

Regression Testing Testing of a previously tested program following modification to ensure that defects have not been introduced in unchanged areas.

Shift Left An approach where testing activities are performed earlier in the software development lifecycle.

System Integration Testing Testing the interfaces and interactions between the system under test and other external systems or services.

System Testing Testing an integrated system to verify that it meets specified requirements and capabilities.

Test Level A specific instantiation of the test process performed against software at a given phase of development.

Test Object The work product that is being tested.

Test Type A group of test activities related to specific quality characteristics (e.g., functional, reliability, security).

White-Box Testing Testing based on an analysis of the internal structure of the component or system (code, architecture, workflows).

STUDENT NOTES & EXAM FLASHCARD REMINDERS (Handwritten Study Space)

4. Realistic ISTQB Practice Exam Questions & Detailed Rationales (Part 1)

Question 1 [K2 - FL-2.1.1]: How does testing in an Agile development model typically differ from testing in a sequential V-model?

A) Agile testing produces more comprehensive test documentation than sequential testing.

B) In Agile, dynamic testing is executed at all test levels in each iteration, whereas in sequential models, dynamic testing occurs mainly in later phases.

C) Agile projects do not require regression testing, whereas sequential projects automate all regression tests.

D) In sequential models, testers perform only non-functional testing, while developers perform functional testing.

Correct Answer: B
Rationale: In iterative/Agile models, each iteration produces a working increment, meaning static and dynamic testing occur at all test levels in every iteration (Section 2.1.1). Sequential models defer dynamic testing until executable code is built in later phases.

Question 2 [K1 - FL-2.1.2]: Which of the following is a recognized GOOD TESTING PRACTICE that applies to ALL software development lifecycles?

A) Test execution must be completely automated before coding starts.

B) Different test levels must have identical test objectives to ensure complete overlap.

C) For every software development activity, there is a corresponding test activity.

D) Non-functional testing should only be performed during User Acceptance Testing.

Correct Answer: C
Rationale: Section 2.1.2 explicitly identifies that a universal practice across all SDLCs is: "For every software development activity, there is a corresponding test activity, so that all development activities are subject to quality control."

Question 3 [K1 - FL-2.1.3]: In which test-first approach are test cases written in a simple natural language using the Given/When/Then format?

A) Test-Driven Development (TDD)

B) Behavior-Driven Development (BDD)

C) Acceptance Test-Driven Development (ATDD)

D) Feature-Driven Development (FDD)

Correct Answer: B
Rationale: BDD (Behavior-Driven Development) expresses the desired behavior of an application with test cases written in natural language using the Given/When/Then format (Chelimsky 2010, Section 2.1.3).

Question 4 [K2 - FL-2.1.5]: Which of the following actions is an example of applying the "SHIFT LEFT" approach in testing?

A) Postponing performance testing until the entire system is deployed to production.

B) Reviewing requirements and user stories from a tester's perspective before code implementation begins.

C) Replacing all automated regression testing with manual user acceptance testing.

D) Having developers execute system integration testing without requirements.

Correct Answer: B
Rationale: Section 2.1.5 lists reviewing specifications from the perspective of testers early in the SDLC as a prime example of achieving shift left to detect defects before code is written.

4. Realistic ISTQB Practice Exam Questions & Detailed Rationales (Part 2)

Question 5 [K2 - FL-2.2.1]: Which test level focuses specifically on testing the interactions between the system under test and other external systems and cloud services?

A) Component Testing

B) Component Integration Testing

C) System Integration Testing

D) User Acceptance Testing

Correct Answer: C

Rationale: Section 2.2.1 states: "System integration testing focuses on testing the interfaces of the system under test and other systems and external services." Component integration tests interfaces between internal components.

Question 6 [K2 - FL-2.2.2]: Which of the following statements about test types is TRUE?

A) Functional testing evaluates how well the system behaves, while non-functional testing evaluates what it does.

B) Non-functional testing can only be conducted during system testing.

C) Black-box, white-box, functional, and non-functional testing can all be applied at all test levels.

D) White-box testing derives test cases from user stories and business requirements.

Correct Answer: C

Rationale: Section 2.2.2 explicitly affirms: "All the four above mentioned test types can be applied to all test levels, although the focus will be different at each level."

Question 7 [K2 - FL-2.2.3]: A tester executes a test case that previously failed to confirm that a bug fix resolved the issue. Simultaneously, an automated test suite executes across the entire application to ensure unchanged modules were not corrupted. What types of testing are being performed?

A) Regression testing followed by maintenance testing.

B) Confirmation testing and regression testing, respectively.

C) System integration testing and alpha testing.

D) Non-functional testing and confirmation testing.

Correct Answer: B
Rationale: Testing to confirm that an original defect was fixed is *Confirmation Testing (re-testing)*. Running tests to ensure no adverse consequences affected unchanged code is *Regression Testing* (Section 2.2.3).

Question 8 [K2 - FL-2.3.1]: An enterprise migrates its customer database from an on-premise Oracle server to an AWS cloud-hosted PostgreSQL database. Which type of testing is triggered, and what is its primary focus?

A) Component testing to rewrite database algorithms.

B) Maintenance testing triggered by an upgrade/migration, focusing on tests of the new environment and data conversion.

C) Retirement testing to delete legacy customer data permanently.

D) System integration testing focused exclusively on user interface styling.

Correct Answer: B
Rationale: Operational environment migrations/upgrades are a primary trigger for maintenance testing, requiring testing of the new environment and data conversion/migration validation (Section 2.3).

Part 2: Additional Questions for Remaining Learning Objectives

Question 9 [K2 - FL-2.1.4]: Which of the following is a typical impact of DevOps on testing?

A) Automated regression tests in the CI/CD pipeline give fast feedback on quality and reduce the risk of regression defects.

B) Because deployment is continuous, non-functional characteristics no longer need to be tested.

C) Test automation is no longer necessary because the pipeline tests itself.

D) Testing is moved to the very end of the pipeline so that development is never slowed down.

Correct Answer: A
Rationale: DevOps benefits for testing: fast feedback on quality, CI promoting shift-left, stable test environments through CI/CD automation, more focus on non-functional characteristics (e.g. performance, reliability), less repetitive manual testing and a lower regression risk. Risks: the delivery pipeline must be defined and established, CI/CD tools must be introduced and maintained, and test automation needs extra resources and can be hard to build and maintain (Section 2.1.4).

Question 10 [K2 - FL-2.1.6]: How can retrospectives be used as a mechanism for process improvement?

A) Management alone holds them before the project starts in order to assign blame for earlier failures.

B) They automatically re-run all regression tests to compare the results of two iterations.

C) The team discusses what was successful and what could be improved, agrees how to incorporate improvements, and records the results (for example in the test completion report).

D) They replace test completion reports and are never documented.

Correct Answer: C
Rationale: Retrospectives are held at the end of a project or iteration, at a release milestone, or when needed. Participants discuss what to retain, what to improve and how to incorporate improvements; the results should be recorded, typically in the test completion report. Benefits for testing: higher test effectiveness and efficiency, better testware quality, team bonding and learning, better test basis quality and better cooperation between development and testing (Section 2.1.6).